

SEMANA 3

Trabajo práctico 03 - API HTTP con Express

1. Información y entrega

- **Modalidad:** individual.
- **Dominio obligatorio:** instrumentos musicales.
- **Entrega:** URL de un repositorio de GitHub.
- **Vencimiento:** lunes siguiente a las 19:59.
- **Ejecución esperada:** `npm install` y luego `npm start` .

Este trabajo práctico es independiente del Gestor de Superhéroes. La primera entrega del proyecto integrador tiene su propia consigna y un alcance menor.

2. Consigna

Crear una API HTTP con Express que administre temporalmente un catálogo de instrumentos musicales.

La aplicación debe:

1. cargar los datos iniciales desde JSON antes de iniciar el servidor;
2. responder una bienvenida;
3. listar todos los instrumentos;
4. filtrar opcionalmente por familia;
5. devolver el detalle de un instrumento por identificador;
6. crear un instrumento en memoria;
7. utilizar códigos de estado coherentes;
8. documentar pruebas exitosas y fallidas.

No se debe copiar la API de series o recetas reemplazando únicamente sustantivos. Los registros, mensajes y decisiones de presentación deben ser propios.

3. Estructura obligatoria

```
tp-03-api-instrumentos/  
|-- datos/  
|   |-- instrumentos.json  
|-- src/  
|   |-- archivos.js  
|   |-- index.js  
|-- .gitignore  
|-- package.json  
|-- package-lock.json  
`-- README.md
```

No se requieren routers, controladores ni servicios separados. Se pueden agregar archivos cuando tengan una responsabilidad clara, pero no es un criterio de esta entrega.

4. Configuración con NPM

Inicializar e instalar:

```
npm init -y
npm install express
```

Configurar:

```
"type": "commonjs",
"scripts": {
  "start": "node src/index.js",
  "check": "node --check src/index.js && node --check src/archivos.js"
}
```

Express debe aparecer en `dependencies` . La entrega debe incluir `package.json` y `package-lock.json` , pero no `node_modules` .

5. Datos iniciales

Crear `datos/instrumentos.json` con al menos cinco instrumentos. Los identificadores deben ser numéricos, únicos y estar ordenados de menor a mayor.

Cada instrumento debe contener:

Propiedad	Tipo	Requisito
<code>id</code>	Número	Único
<code>nombre</code>	Texto	Nombre del instrumento
<code>familia</code>	Texto	Por ejemplo cuerda, viento o percusión
<code>origen</code>	Texto	Lugar o contexto de origen
<code>descripcion</code>	Texto	Descripción breve
<code>disponible</code>	Booleano	<code>true</code> o <code>false</code>

Ejemplo de forma, que no debe reutilizarse como registro de la entrega:

```
{
  "id": 1,
  "nombre": "Instrumento de ejemplo",
  "familia": "Familia de ejemplo",
  "origen": "Origen de ejemplo",
  "descripcion": "Descripción utilizada solamente para mostrar la estructura.",
  "disponible": true
}
```

6. Carga asíncrona

`src/archivos.js` debe exportar una función asíncrona que:

- lea un archivo mediante `node:fs/promises` ;
- utilice codificación `utf8` ;

- convierta el texto mediante `JSON.parse` ;
- devuelva el valor interpretado.

`src/index.js` debe:

- construir la ruta con `node:path` y `__dirname` ;
- esperar la lectura dentro de `main` ;
- crear e iniciar Express solamente después de disponer de los datos;
- informar un error de inicio si el archivo no puede leerse o interpretarse.

7. Contrato obligatorio

Bienvenida

```
GET /
```

Debe responder `200` y un objeto JSON que indique que la API está disponible.

Listado y filtro

```
GET /api/instrumentos
GET /api/instrumentos?familia=cuerda
```

Sin consulta, debe devolver todos los registros. Con `familia` , debe utilizar `filter` y comparar sin distinguir mayúsculas de minúsculas.

Si no existen coincidencias, debe responder:

- estado `200` ;
- arreglo vacío `[]` .

El filtro no debe modificar la colección original.

Detalle

```
GET /api/instrumentos/:id
```

Debe:

- obtener `req.params.id` ;
- convertirlo a número;
- buscar con `find` ;
- responder `200` y el instrumento cuando existe;
- responder `404` y un objeto con `error` cuando no existe.

Creación en memoria

```
POST /api/instrumentos
```

Antes de las rutas debe configurarse:

```
app.use(express.json());
```

El cuerpo debe incluir `nombre` , `familia` , `origen` , `descripcion` y `disponible` . El identificador será generado por el servidor a partir del último registro.

La validación debe considerar que `false` es un valor válido. Por eso no se debe comprobar disponibilidad mediante `!disponible` ; se debe distinguir específicamente si el valor es `undefined` .

Si falta un campo obligatorio:

- responder `400` ;
- devolver un objeto con `error` ;
- no agregar el instrumento.

Si el cuerpo está completo:

- agregar el recurso al arreglo;
- responder `201` ;
- devolver el instrumento creado, incluido su identificador.

8. Memoria y persistencia

Después de un POST exitoso, el nuevo instrumento debe aparecer en el listado mientras el servidor continúe activo.

Al reiniciar el proceso, el instrumento desaparecerá porque la aplicación vuelve a cargar el JSON original. No se debe escribir el cambio en `instrumentos.json` ni incorporar una base de datos.

El README debe explicar esta limitación.

9. Matriz de pruebas obligatoria

Probar con el cliente HTTP elegido:

Caso	Solicitud	Estado esperado
Bienvenida	GET /	200
Listado	GET /api/instrumentos	200
Filtro con coincidencias	GET /api/instrumentos?familia=...	200
Filtro sin coincidencias	GET /api/instrumentos?familia=inexistente	200
Detalle existente	GET /api/instrumentos/1	200
Detalle inexistente	GET /api/instrumentos/999	404
Creación válida	POST /api/instrumentos	201
Creación sin nombre	POST /api/instrumentos	400
Creación con <code>disponible: false</code>	POST /api/instrumentos	201

Para POST utilizar:

```
Content-Type: application/json
```

Ejemplo de cuerpo válido que debe reemplazarse con datos propios:

```
{
  "nombre": "Nuevo instrumento",
  "familia": "Percusión",
  "origen": "Origen propio",
  "descripcion": "Descripción propia para probar la creación.",
  "disponible": false
}
```

Después de crear:

1. repetir el listado;
2. localizar el recurso nuevo;
3. detener el servidor;
4. iniciarlo nuevamente;
5. comprobar que el recurso ya no existe.

10. README

El `README.md` debe incluir:

```
# Trabajo práctico 03

## Descripción

## Instalación

## Ejecución

## Endpoints

## Ejemplos de solicitudes

## Códigos de estado

## Persistencia de los datos
```

Debe explicar:

- `npm install` y `npm start` ;
- cómo detener el servidor;
- método y URL de cada endpoint;
- cuerpo necesario para POST;
- casos `200` , `201` , `400` y `404` ;
- diferencia entre parámetro de ruta y consulta;
- función de `express.json()` ;
- por qué las creaciones desaparecen al reiniciar.

11. Criterios de evaluación

Criterio	Puntaje
Proyecto NPM, Express y carga asíncrona	1,5 puntos
Datos JSON válidos y propios	1 punto
Bienvenida y listado	1 punto
Filtro mediante query	1,5 puntos
Detalle mediante parámetro y 404	1,5 puntos
Creación, validación mínima y 201 / 400	2 puntos
Códigos y respuestas coherentes	0,5 puntos
README, .gitignore y repositorio reproducible	1 punto
Total	10 puntos

12. Fuera de alcance

No se evaluarán:

- routers o controladores;
- vistas HTML;
- middleware personalizado;
- PUT , PATCH o DELETE ;
- persistencia después del reinicio;
- base de datos;
- validaciones avanzadas;
- autenticación o seguridad.

Agregar funcionalidades fuera de alcance no compensa un endpoint obligatorio incompleto.

13. Comprobación y entrega

Antes de enviar la URL:

- ejecutar `npm run check` ;
- ejecutar `npm start` ;
- completar la matriz de pruebas;
- comprobar `disponible: false` ;
- verificar que Express figure en `dependencies` ;
- verificar que `package-lock.json` esté publicado;
- confirmar que `node_modules` no esté en GitHub;
- leer el README como si se instalara en otra computadora;
- restaurar cualquier cambio temporal utilizado para provocar errores.